Сервис прототипов в production

Что подготовить, какие кнопки нажать
и как убедиться, что пользователь получил результат.

Подготовлено 12 октября 2026 · без значений секретов

Суть запуска

Разворачиваем отдельный исполнитель задач — worker. Он работает рядом с API ChatRouter и использует уже существующие БД, очередь, хранилище файлов и Gateway. Новый публичный адрес worker не нужен.

Два режима — разная степень готовности

Текущий режим

Worker + сборка в Gateway

Worker обращается к выбранной модели и передаёт подготовленный код в Gateway для сборки.

`direct-react-v1`

Контейнер и конфигурация подготовлены. Полный production-путь пользователя ещё предстоит принять.

Следующий этап

OpenCode внутри Gateway

Агент пишет код в sandbox и вызывает модель только через Model Proxy в API.

`release-lock.agent = null`

Сетевая изоляция Gateway не принята. Этот режим текущим запуском не включается.

Что уже подтверждено

- Локальный worker-контейнер запускается; проверки очереди, хранилища и сообщения о готовности прошли.
- 9 октября production API выдал короткий токен задачи собственной тестовой задаче; discovery и отзыв токена прошли.
- Отдельный OpenCode image запустился в Gateway, но выявлены проблемы пользователя процесса и исходящей сети.

Состояние на дату документа

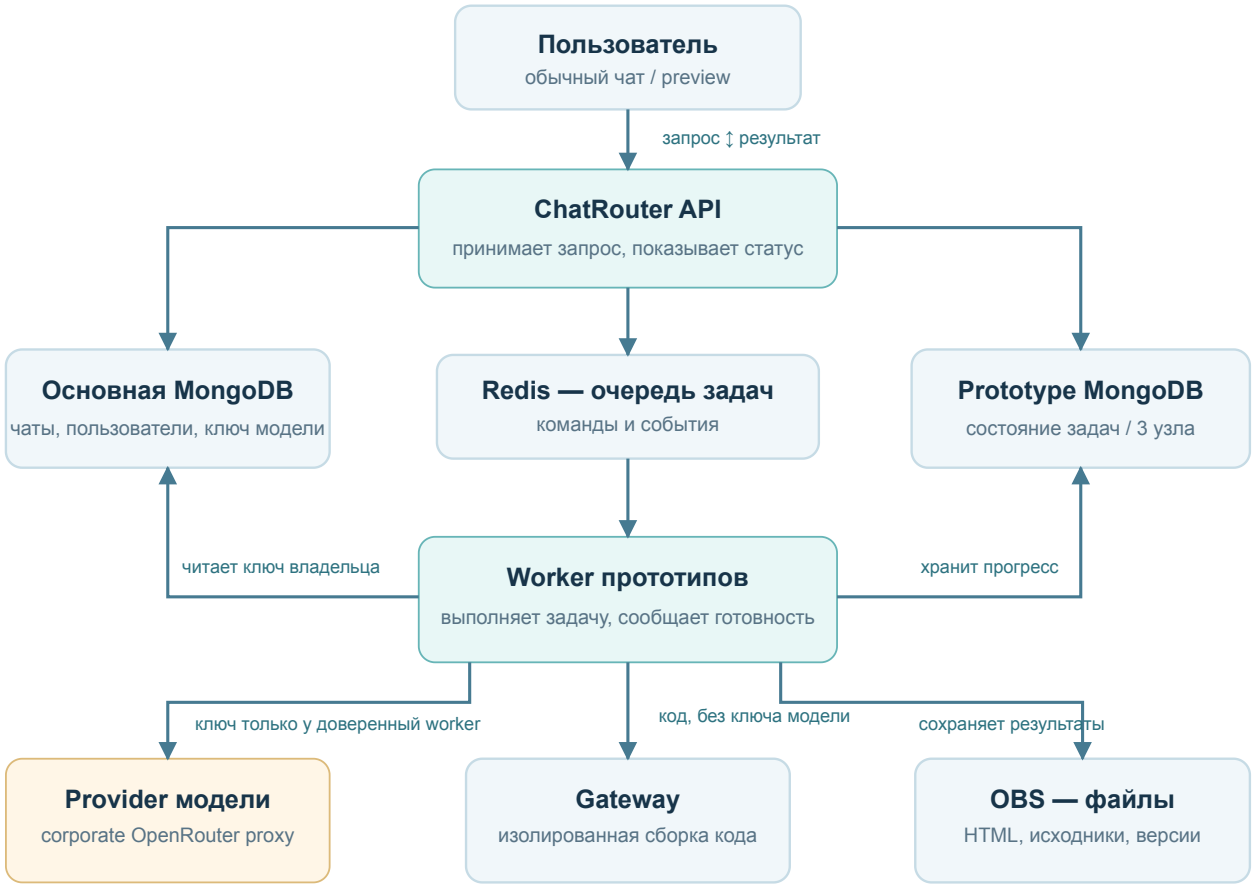
Код и файлы проверены 12 октября. Сетевые и production-результаты относятся к 8–9 октября; сегодня их заново не проверяли. Запущенный pod ещё не означает готовый продукт.

Кто участвует

DevOps — секреты, БД, CI и запуск. **Gateway** — изоляция удалённой среды. **Команда ChatRouter** — API/worker и проверки. **Владелец пилота** — аккаунт, модель и допустимая стоимость.

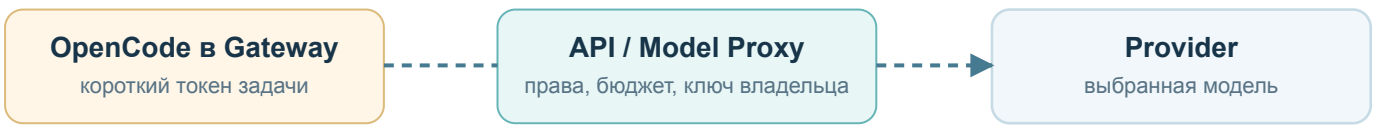
Кто с кем общается

Сплошные линии — текущий режим worker.
Будущий OpenCode-путь показан отдельно ниже.



API читает состояние и файлы, возвращает карточку, preview и скачивание.

Будущий путь модели с OpenCode



Этот путь отдельно принимается; текущий worker deployment его не включает.

> **Две БД — разные задачи**
>
> Основной `MONGO_URI` сохраняется. API и worker используют один `PROTOTYPE_MONGO_URI` отдельного replica set; наблюдавшийся production store: `openrouter_chat`, `rs0`.

Что должно быть готово

Проверить этот список до активного включения генераций.

Что	Что проверить	Ответственный
Версия кода	В GitLab есть worker Dockerfile, chart и include `.gitlab/prototype-service.yml`. Локальные файлы сами не создают CI-кнопки.	ChatRouter
CI и registry	Runner linux/amd64 умеет публиковать image; скрытый `kanikocfg` настроен. Deploy runner имеет права в нужном кластере.	DevOps
Основная БД	Существующие пользователи и ключи доступа к модели доступны. Пользователей/историю в БД прототипов не копировать.	DevOps / API
Prototype DB	Отдельный replica set; доступен primary и все 3 адреса из API/worker. API и worker смотрят в один store.	DevOps
Redis	Очередь сохраняется; при нехватке памяти задачи не удаляются. Worker проверяет требуемые настройки.	DevOps
OBS	Bucket доступен для записи и чтения, результаты нельзя случайно перезаписать. Нужны права на тестовые собственной тестовой папкой операции.	DevOps
Сеть	Worker видит Gateway, API и corporate provider. Для OpenCode нужен отдельный ограниченный agent profile.	Gateway / DevOps
Secrets	Есть `librechat-secret-env` для API и `prototype-worker-env` для worker; только нужные каждому поля.	DevOps
Рабочая папка	Кластер выделяет 5Gi через `csi-disk-topology`. Это рабочий кэш; важное состояние остаётся в Mongo/OBS.	DevOps
Тестовый владелец	Модель выбрана в чате, есть действующий ключ и средства. Функция включена в YAML и доступна авторизованным пользователям при готовом сервисе.	Владелец пилота

> **Production URI — внутренний адрес кластера**
>
> `127.0.0.1` из SSH-туннеля нужен для Compass и локальной проверки. В production pod он укажет на сам pod и не заменяет внутренний Mongo URI.

Доступ к двум существующим системам не значит две копии данных

Основная MongoDB хранит аккаунт и ключ модели. Prototype MongoDB хранит задачи и прогресс. Worker использует обе по назначению; основной чат не переносится в replica set прототипов.

Порядок deployment

Кнопки worker доступны в MR в `main` и на `main`. Обычный pipeline не ждёт их ручного запуска.

1. **Подготовить нужную версию и Secrets**

   Убедиться, что изменения есть в GitLab pipeline. Оператор заранее создаёт `prototype-worker-env`, сверяет API Secret, БД и ключи. Значения не вставлять в Git/CI log.

2. **Обновить ChatRouter API**

   Существующий путь: `build → deploy_GK_prod`. Нужны backend guards и worker routes. Порядок Задача API определяется текущим pipeline; это не новая manual-кнопка worker.

3. **Проверить настройки API**

   `prototypeAgent.enabled=true`, `PROTOTYPE_TRANSPORT_ENABLED=true`, auth bearer; `OPENROUTER_API_URL` — корпоративный provider. Для adapter: `PROTOTYPE_MODEL_PROXY_ENABLED=true`, RSA key только API.

4. **Нажать build_prototype_worker**

   Job собирает отдельный image и передаёт deploy его точный digest. Artifact хранится 7 дней; после истечения надо пересобрать.

5. **Сначала развернуть без обработки задач**

   В форме manual `deploy_prototype_worker` явно задать `WORKER_INTAKE_ENABLED=false`. Процесс проверяет зависимости и не запускает генерации.

6. **Проверить режим ожидания**

   `/healthz → 200`; application `/readyz → 503` ожидаем. В режиме ожидания chart использует healthz для готовности pod, поэтому Задача CI может быть успешным.

7. **Включить контролируемый пилот**

   Повторить deploy с `WORKER_INTAKE_ENABLED=true`. После проверок Redis/OBS worker сообщает API готовность; `/readyz → 200`. Текущий лимит — до $2 на один ход прототипа.

8. **Пройти пользовательский цикл**

   Один разрешённый аккаунт и согласованная модель: план, подтверждение, preview, правка, итог, скачивание. Проверить стоимость и отмену.

> **Сейчас по умолчанию обработка включена**
>
> В worker values и CI стоит `true`. Без явного `false` deploy включает приём задач и может вызвать платные обращения. Перед включением обработки согласовать объём пилота и стоимость.

Где: namespace `openrouter-chat`, release `prototype-worker`; values `envs/prod-prototype-worker.yaml`; chart `helm/prototype-service`. Job создаёт Secret для скачивания image, но не рабочий env Secret. При ошибке активного запуска Helm выполняет откат; это не удаляет данные задачи.

Один источник, разные получатели

Одинаковые значения у API и worker допустимы.
Две независимые ручные копии — источник ошибок.

Где найдено	Что это и что с ним делать
`.env` в корне	Настройки локальной разработки. Локальный worker читает этот файл. Отдельного `.env` внутри `apps/prototype-service` сейчас нет.
`.prototype-dev/runtime.json`	Автоматическая запись для связи локальных API/worker. Повторяет Redis/БД прототипов и содержит временную identity. Не редактировать как конфигурацию; в production не переносить.
`.prototype-dev/production-identity/adapter-secrets.env`	Защищённый файл проверки production через SSH-туннель. Это отдельное окружение. Не объединять с локальным .env и не копировать в pod.
`worker.production.example.env`	Шаблон имён и placeholders. Здесь нет настоящих секретов. На его основе оператор готовит worker Secret.
`envs/prod.yaml`	Настройки API в Git. В нём обнаружены непустые значения `CREDS_KEY`, `CREDS_IV`, `JWT_SECRET`, `JWT_REFRESH_SECRET`. Их следует вынести из YAML/Git.

Правильная схема production

> **Управляемое хранилище DevOps → два ограниченных набора**
>
> API получает `librechat-secret-env`. Worker получает `prototype-worker-env`. Общие значения выдаются из одного источника и обновляются согласованно; полный API Secret worker не передаётся.

Как вынести значения без поломки

1. Сохранить актуальные значения в защищённом production источнике.
2. Проверить, что API и worker получают нужные одинаковые поля.
3. Убрать значения из YAML в Git и проверить запуск и расшифровку существующего ключа модели.
4. Если значения в Git действующие, оператор оценивает доступ и планирует безопасную замену ключей. Она выполняется отдельно от переноса.

> **CREDS_KEY / CREDS_IV нельзя просто заменить**
>
> Ими зашифрованы сохранённые ключи доступа к модели. Для замены нужен план расшифровки старых записей и миграции. Смена JWT keys также может завершить пользовательские сессии.

В этой работе секреты не переносились и не менялись. Файлы и текущие процессы сохранены. В PDF присутствуют только имена переменных.

Кому какие настройки нужны

Таблица относится к текущему режим workery. Gateway sandbox не получает долговременные ключи.

Группа	API	Worker	Sandbox
`MONGO_URI`	Да	Читает ключ владельца	Нет
`PROTOTYPE_MONGO_URI`	Да	Да, та же БД	Нет
`REDIS_URI`	При включённой очереди	Да	Нет
`AWS_*` / OBS	Да	Да	Нет
`PROTOTYPE_SERVICE_TOKEN`	Да	Тот же токен	Нет
`CREDS_KEY / CREDS_IV`	Да	Те же значения	Нет
`SANDBOX_GATEWAY_TOKEN`	Не нужен этому пути	Да	Нет
Закрытый ключ подписи RSA	**Только API**	Нет	Нет
`JWT_SECRET / JWT_REFRESH_SECRET`	**Только API**	Нет	Нет
Ключ доступа к модели	В основной БД	Читает для выбранного владельца	Нет

Минимальный список worker Secret

`MONGO_URI`, `PROTOTYPE_MONGO_URI`, `REDIS_URI`, `SANDBOX_GATEWAY_URL/TOKEN`, `PROTOTYPE_SERVICE_TOKEN`, `OPENROUTER_API_URL`, `CREDS_KEY/IV`, `AWS_BUCKET_NAME`, `AWS_REGION`, `AWS_ENDPOINT_URL`, `AWS_FORCE_PATH_STYLE`, `AWS_ACCESS_KEY_ID/SECRET_ACCESS_KEY`.

Адреса, bucket, region и флаги — настройки, не всегда секреты. Пароли и ключи оператор хранит защищённо. Реальные значения берутся из существующей среды, а не из примера PDF.

> **RSA подпись — один источник на API**
>
> `PROTOTYPE_MODEL_TOKEN_PRIVATE_KEY_BASE64` или `PROTOTYPE_MODEL_TOKEN_PRIVATE_KEY_PATH`, не оба одновременно. Будущий agent получает короткий токен задачи, а не RSA private key.

Как запускается процесс

```
# Production: настройки уже переданы кластером
node apps/prototype-service/src/entrypoint.js

# Development: читает корневой .env
npm run prototype:dev
```

Production entrypoint не загружает корневой .env. Image не должен содержать root env/файлы секретов для проверок. Если нет роли или обязательного поля, при запуске будет указано имя отсутствующей настройки.

От запроса до прототипа

Всё происходит в обычном чате.
Worker и сборка работают за карточкой задачи.



Если worker недоступен

Новая генерация не начинается. Обычный чат работает; сохранённые результаты доступны, свои задачи можно отменить.

Если нет ключа или средств

Исправить доступ/баланс и продолжить через разрешённое действие. Не создавать скрытых автоматических повторов.

Подготовка и уточнения могут обращаться к платной модели ещё до подтверждения плана. Показывать фактическую стоимость; не считать её нулевой только потому, что сборка ещё не началась.

Когда запуск можно считать успешным

Процесс готов

- При обработке задач: `/readyz → 200`.
- API получает свежие сообщения готовности worker.
- Очередь/OBS/prototype Mongo работают.
- Секретов в log/архивах нет.

Пользовательский цикл принят

- Инструмент виден только нужному аккаунту.
- План, preview, правка и скачивание работают.
- Стоимость и cancel проверены.
- Обычный чат работает при сбое.

Как остановить новые генерации

1. Отключить prototypeAgent/transport на API.
2. Развернуть worker с `WORKER_INTAKE_ENABLED=false`.
3. Сверить активные задачи и закончить только собственные удалённые sessions. Не удалять Mongo, OBS и Secrets.

Остановка worker не доказывает отмену уже запущенного Gateway sandbox. Если результат операции неизвестен, сначала проверить её состояние; не повторять автоматически оплачиваемое действие.

Что ещё нужно для OpenCode

> **Gateway должен принять отдельную изоляцию агента**
>
> 9 октября session запускалась под root вопреки USER образа; provider и посторонний интернет были доступны. Наш launcher исправляет UID только нашего процесса. Платформе нужен запуск от обычного пользователя и запрет лишних сетевых направлений.

После исправления сети повторить проверки, принять короткие токены, потоковые ответы, инструменты и учёт стоимости и работу установленного OpenCode с исходниками. Только после этого закрепить принятый образ агента. Для Storybook 8 октября корпоративный endpoint давал 307 на себя; legacy тогда прошёл. Разрешать выбранный MCP origin отдельным согласованным исключением.

Файлы, с которых начинать

Worker CI	`.gitlab/prototype-service.yml`
API / worker values	`envs/prod.yaml` `envs/prod-prototype-worker.yaml`
Deployment / env example	`helm/prototype-service` `apps/prototype-service/deploy/gateway/worker.production.example.env`
Подробности / Gateway	`docs/integrations/prototype-agent/gateway-worker-production-runbook.md` `docs/integrations/prototype-agent/gateway-agent-isolation-report-2026-10-09.md`

Простой словарь: worker — исполнитель задач; pod — процесс в кластере; Secret — защищённые настройки; image — пакет приложения; digest — его точный идентификатор; Redis — очередь; OBS — файлы; provider — сервис, который отвечает моделью.